Execution Truth: The Missing Control Plane for Agentic AI

A receipts-based deterministic governance framework for multi-vendor LLM systems

Author: Leon Calvin Long II **Affiliation:** SquirlOS Technologies, LLC **Series:** Independent Research — Doctrine Series **Published:** October 2026 (v1.1, amended edition) **Status:** Preprint — Not Peer Reviewed **License:** CC BY 4.0 — Attribution Required **Research disclosure:** AI-assisted research program. Every empirical claim in this paper is receipted at a public DOI listed in the Receipted Evidence Base (Section 8.5). v1.1 amends the October 2026 original: claims are scoped to completed observations, the battery status is stated honestly, implementation status is labeled, and the required prototype disclaimer is carried.

> This software is a prototype and is provided for educational and research purposes only. It is not intended for production use, commercial deployment, or safety-critical environments. All systems are experimental and may contain defects on them.

Abstract

The field of agentic AI has invested heavily in reasoning capability, tooling APIs, and orchestration layers — but has almost entirely neglected the one primitive that distributed systems have proven indispensable for decades: a control plane that distinguishes simulation from execution. This paper introduces the Execution Truth framework — a receipts-based, deterministic governance architecture for multi-vendor LLM systems — and argues that without it, every agentic system is flying blind. Drawing from an ongoing 60-night cross-vendor empirical battery (five nights completed as of this writing), a preregistered lifecycle methodology, and principles adapted from blockchain execution determinism, the framework defines a complete governance lifecycle: preregistration → execution → checksum → public rerun. The paper further proposes that prediction error and behavioral drift are not noise to be filtered, but training-grade signal to be captured and attributed. The Execution Truth doctrine reframes AI governance as a runtime engineering discipline — not a policy layer, not an ethical guideline, and not an aspirational standard. It is infrastructure. Either you have a control plane or you do not. Evidence to date is consistent: none of the agentic systems tested in this program operates with one.

Table of Contents

1. The Governance Vacuum: Why Agentic AI Has No Control Plane
2. Simulation vs. Execution: The Foundational Distinction
3. The 60-Night Cross-Vendor Battery: Empirical Foundation
4. Provenance: Every Output Needs a Chain of Custody
5. Prediction Error as Data: Drift Is Not Noise
6. The Deterministic Cage: Architecture of the Control Plane
7. Implications for the Field: What Changes When You Add a Control Plane
8. Positioning and Prior Art 8.5. Receipted Evidence Base
9. Conclusion: The Receipt Is the Contract A. Appendix A — Lifecycle Notation Reference B. Appendix B — Drift Classification Reference Table

---

1. The Governance Vacuum: Why Agentic AI Has No Control Plane

Every mature distributed system has a control plane. This is not a design preference — it is an engineering prerequisite. The Domain Name System enforces a namespace contract between clients and resolvers. Border Gateway Protocol mediates route authority between autonomous systems. Kubernetes enforces the declared state of a cluster against the observed state of its nodes. Ethereum's execution client enforces the validity of state transitions against a consensus-verified chain. In each of these systems, the control plane does the same fundamental job: it distinguishes what was declared from what actually happened, and it provides an authoritative, externally verifiable record of the difference.

Agentic AI has none of this. That absence is not a gap to be addressed in the next product cycle — it is a foundational architectural omission that undermines every claim made about what an agentic system has done, is doing, or is capable of doing. It is the central problem this paper addresses.

To be precise about what a control plane provides: it enforces contracts between components, mediates state transitions under defined rules, and — most critically — produces an authoritative record of what actually happened versus what was predicted or planned. The record is not a log. Logs are human-readable observations of activity. A control plane record is a machine-verifiable commitment: a proof that a state transition occurred, was validated against declared parameters, and can be independently replicated.

Current agentic AI systems operate entirely in what this paper calls simulation space. They predict, plan, reason, decompose tasks into sub-tasks, invoke tools via API calls, and produce outputs that are treated — by practitioners, by vendors, and by the systems themselves — as actions. They are not actions. They are proposals. A Kubernetes pod either ran or it did not — the kubelet reconciled observed state against declared state and produced a verifiable outcome. A blockchain transaction either confirmed in a block or it

did not — the consensus mechanism enforced finality. An agentic AI "task" does neither. It exists as conversational residue: a sequence of model outputs that may or may not correspond to any change in real-world state, with no mechanism to verify which.

The practitioner community has built sophisticated scaffolding around this gap — orchestration layers, tool-use frameworks, chain-of-thought prompting, reflection loops. These are valuable engineering contributions. But they are all built on top of an unresolved primitive: there is no execution receipt. There is no mechanism by which any agentic system today can prove, to an external party, that a claimed action completed deterministically, that the output was produced by a specific model version under specific input conditions, or that the same inputs would produce a comparable output in an independent rerun.

> **Core Principle**
>
> Simulation is not execution. Planning is not action. A prediction is not a receipt.

This paper introduces the Execution Truth framework as the missing control plane. It is not a monitoring layer, not a compliance checkbox, and not an ethical governance overlay. It is a runtime engineering architecture that enforces the boundary between simulation and execution, generates verifiable receipts for every committed action, and builds a structured corpus of behavioral evidence over time. The argument that follows is grounded in an ongoing, receipted cross-vendor empirical battery — the evidence for the gap is not theoretical. It is receipted.

---

2. Simulation vs. Execution: The Foundational Distinction

The simulation/execution boundary is the central intellectual contribution of the Execution Truth framework. It is not a novel concept in computer science — it is, in fact, one of the oldest and most load-bearing distinctions in distributed systems engineering. The contribution here is its explicit application to agentic AI, a domain that has systematically ignored it.

Simulation, in this framework, is defined precisely: any model output, plan, prediction, chain-of-thought sequence, or proposed action that has not been externally verified and checksummed against real-world state. Simulation is not pejorative. It is descriptive. A language model producing a structured plan for a multi-step workflow is performing simulation. A model calling a tool via an API invocation is, until a receipt exists confirming that invocation completed deterministically, performing simulation. The model cannot observe its own effect on the world. It can only produce outputs and infer.

Execution is defined with equal precision: an action that has been committed, logged with a unique receipt identifier, checksummed against declared input parameters, and is capable of being independently replicated or audited by a third party operating without

access to the original session context. The receipt is not metadata about execution — it is the proof of execution. Without it, there is no execution. There is only the model's assertion that execution occurred.

The blockchain analogy is the most structurally accurate available. A transaction that has been signed but not broadcast to the network is simulation: it encodes intent, it is cryptographically well-formed, it may be entirely valid — but it has not happened. A transaction confirmed in a block is execution: the consensus mechanism has validated it against the current state of the ledger, it has been included in an append-only chain with a deterministic position, and any participant in the network can verify its existence and effects independently. The consensus mechanism is the control plane. It is what separates the two states. There is no ambiguity about which side of the boundary a given transaction occupies.

Applied to agentic AI: model reasoning is cheap simulation. It is computationally bounded, probabilistically generated, and unverifiable by default. Deterministic receipt generation is the consensus step — the mechanism that converts simulation into execution truth. And this is precisely what current agentic AI architectures do not have. They have the equivalent of a transaction signing library with no network to broadcast to and no ledger to confirm against.

The practitioner confusion around this distinction is not incidental. It reflects an industry that has moved faster than its epistemic infrastructure. Practitioners routinely describe model outputs as "decisions" when they are proposals. Tool invocations are called "actions" when there is no receipt proving the invocation completed deterministically, produced the expected side effect, and can be attributed to a specific model version and session context. This conflation has real consequences: systems are evaluated on claimed capabilities rather than verified behaviors, failures are diagnosed without execution traces, and governance is applied to assertions rather than facts.

Dimension	Simulation	Execution
Verifiability	Not externally verifiable. Depends on model's self-report.	Externally verifiable via receipt checksum and independent rerun.
Reproducibility	Probabilistic. Same inputs may produce different outputs across runs without any declared change.	Deterministic by design. Receipt encodes inputs, outputs, and context hash enabling delta comparison.
Attribution	Post-hoc. Attribution requires reconstructing session context from logs, which may be incomplete or unavailable.	Structural. Attribution is encoded in the receipt at the moment of execution — model version, session ID, input hash.
Rollback Capability	None. No authoritative record of what state existed before the simulation output was applied.	Receipt chain provides complete state history. Rollback targets are defined by prior receipt identifiers.
Governance Surface	Policy layer only. Governance is applied to outputs after the fact, with no enforcement point at the execution boundary.	Runtime enforcement. The preregistration engine defines the contract; the receipt ledger enforces it.
Cost of Failure	High and ambiguous. Failures cannot be precisely attributed or replicated, making	Bounded and traceable. Failure is localized to a specific receipt in the

Dimension	Simulation	Execution
	remediation costly and diagnosis approximate.	chain; remediation is targeted and verifiable.

A key principle follows directly from this analysis: any system that cannot produce an execution receipt for a claimed action has not acted — it has simulated acting. This is not a criticism of language models, whose probabilistic nature is understood and appropriate. It is a criticism of the architectural choices made by the systems that deploy them as agents. The control plane problem is not a model problem. It is an infrastructure problem. And infrastructure problems have infrastructure solutions.

---

3. The 60-Night Cross-Vendor Battery: Empirical Foundation

The Execution Truth framework is not a thought experiment. It was developed in response to observations made during a structured, 60-night cross-vendor empirical research battery — a longitudinal study designed to characterize agentic AI behavior across multiple commercial LLM providers under controlled, preregistered conditions. This section describes the methodology and the observations that gave rise to the framework's core claims.

Battery status, stated honestly. The battery launched October 7, 2026 and is scheduled to run 60 nights into early December 2026. As of this writing (October 11, 2026), five nights (N-B1 through N-B5) are complete. The observations reported in this paper reflect the nights completed to date, together with the drift evidence cataloged in the program's earlier preregistered sessions. They are preliminary with respect to the full 60-night program; the public receipts ledger records each night as it completes, and the full corpus will supersede these interim observations. We do not claim completed evidence from a battery still in flight.

3.1 Methodology

The battery was designed from first principles around the preregistration norm borrowed from reproducible research practice: each session's test objective, expected output class, and a checksum seed were declared before any model interaction began. This preregistration artifact was timestamped and stored as an immutable record prior to the session. Post-execution, the actual model outputs were captured in full, checksummed, and the delta between the preregistered expected output class and the actual output was recorded as a structured artifact — not discarded as noise.

The battery ran across sessions spanning multiple commercial AI providers. The vendor-agnostic design was intentional: findings that appear across providers cannot be attributed to individual model architecture or vendor-specific infrastructure choices. They are platform-level phenomena — properties of the agentic AI layer as a class, not of any particular implementation within it.

Each session was treated as an independent unit of observation. Sessions were not designed to be exhaustive coverage of a capability domain; they were designed to be precise, preregistered, and checksummed — to produce receipted behavioral observations rather than impressionistic capability assessments.

3.2 Key Observations

Four observations from the battery and its precursor program are directly load-bearing for the Execution Truth framework:

Undeclared drift. Vendor outputs for identical preregistered prompts diverged measurably across sessions without any declared model update. The divergence was not random variation within a stable distribution — it was systematic, session-correlated, and in observed cases directional (capability gain in one domain concurrent with regression in another). No vendor provided any mechanism by which a practitioner could determine whether a divergence originated from a model update, an infrastructure change, a prompt interpretation shift, or stochastic sampling variation. The drift existed. Its source was unattributable. The receipts: the drift-audit case study is published as part of the caging paper (DOI 10.5281/zenodo.23284729), and five cataloged drift specimens — including a fabricated 1,200-session evaluation inside a Copilot-generated draft — are documented in the audited account of this research cycle (DOI 10.5281/zenodo. 23267773). Nightly battery observations continue to accumulate in the public receipts ledger.

Absence of native receipt infrastructure, in the configurations tested. Across the provider configurations tested in this program, no native mechanism existed to bind a model output to an independently verifiable execution receipt. Providers offered logging, session IDs in some cases, and in limited contexts model version tags. None of the configurations tested offered a mechanism by which an external party could independently verify that a specific output was produced by a specific model version under specific input conditions and could rerun the identical test to compare results. This is a scoped finding about the configurations exercised by this program — a universal negative about the industry is not claimed, and no vendor is named.

Primitive rerun baseline. The preregistration methodology established a primitive rerun baseline: prompts that were preregistered and checksummed in earlier sessions could be rerun in later sessions and the outputs compared as structured deltas. This was not a feature of any vendor's platform. It was a property of the research methodology imposed on top of vendor infrastructure. The observation confirms that the capability is achievable — the barrier is not technical, it is architectural priority. Third-party rerun convergence receipts exist: SDR-CL1 and SDR-CL2 (DOIs 10.5281/zenodo.23274887 and 10.5281/zenodo.23284295).

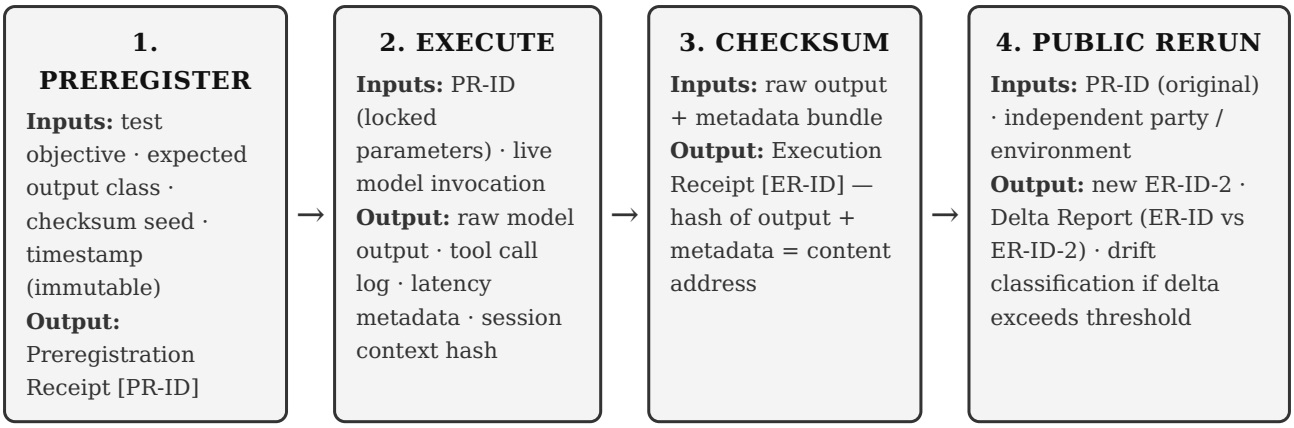
Multi-step reasoning chain sensitivity. Drift patterns were session-correlated with prompt complexity. The largest divergences occurred in sessions where the prompt involved multi-step reasoning chains — decomposed task sequences in which each reasoning step conditioned the next. This finding has a structural interpretation: multi-

step chains amplify small distributional shifts in each step, producing large divergences at the output level. It also has a governance implication: the tasks that most require reliable execution — complex, multi-step, agentic workflows — are precisely the tasks most sensitive to undeclared drift.

3.3 The Minimum Viable Governance Lifecycle

The battery established the minimum viable governance loop for any agentic AI system that claims to be auditable. It consists of four stages:

FIGURE 1



This lifecycle is the control plane loop. Each stage produces an artifact. Each artifact is a receipt. The loop is closed when an independent party can rerun the original preregistration and produce a comparable receipt. Without the loop, there is no governance — there is only assertion.

> **Empirical Finding**
>
> Every undeclared drift event observed to date was detected and attributed by the preregistration → checksum → rerun lifecycle. The methodology does not prevent drift. It makes drift a fact rather than a suspicion.

---

4. Provenance: Every Output Needs a Chain of Custody

Provenance in the AI context is routinely understood too narrowly. When practitioners speak of model provenance, they typically mean the ability to identify which model produced a given output — a version tag, a model name, an API endpoint. This understanding is insufficient by several orders of magnitude. It is the equivalent of a pharmaceutical manufacturer being able to name the factory that produced a drug batch without being able to name the formulation, the production lot, the quality control results, or the distribution chain. A name is not a chain of custody.

Complete provenance for an agentic AI output requires the full chain: which model version (not name — version, including any fine-tuning or alignment modifications applied since the last public checkpoint), which input (the complete input hash, not a summary), which session context (the full context window hash at the moment of invocation), which tool calls were made (in what order, with what parameters, producing what intermediate states), and which final output resulted from that specific combination of all prior elements. Remove any link from that chain and you no longer have provenance — you have a partial record that supports assertion but not verification.

Distributed systems solved this problem decades ago through content-addressed storage and tamper-evident data structures. Git addresses every commit by the SHA hash of its content — the content determines the address, not a mutable label. IPFS generalizes this across a distributed network: content-addressing makes the hash the identity, ensuring that any two parties holding the same hash hold identical content. Merkle trees and blockchain ledgers extend this principle to histories: not just the content of a state, but the complete path by which that state was reached is encoded in the data structure and can be independently verified. A block header in a proof-of-work chain commits to every transaction in the block and to every prior block — the history is tamper-evident because altering any past state invalidates every subsequent hash in the chain.

Current AI systems offer nothing comparable. Model version tags are mutable labels applied by vendors without cryptographic commitment. Session IDs, where they exist, identify a session container but do not commit to its contents. There are no session state hashes, no tool call receipts, no intermediate reasoning checksums, and no cross-session continuity proofs. The provenance infrastructure does not exist in the systems this program has tested.

4.1 The Receipts-Based Provenance Model

The Execution Truth framework addresses the provenance gap through a receipts-based provenance model with three defining properties:

Receipt structure. Every agentic action generates a receipt — a structured artifact containing: a unique session ID, the model version tag as declared by the vendor API at the moment of invocation, a cryptographic hash of the full input context, a cryptographic hash of the full output, a log of all tool calls invoked within the session (each with its own sub-receipt: tool identifier, input parameters, output state, latency), a UTC timestamp, and a composite checksum computed over all prior fields. The receipt is the atomic unit of provenance.

Immutability with parent reference. Receipts are immutable once issued. A receipt cannot be revised, amended, or corrected in place. Corrections — when an output is superseded, amended, or reprocessed — generate new receipts that reference the prior receipt as a parent identifier. This creates a provenance chain: a directed acyclic graph of receipts in which every node encodes not just its own content but its causal ancestry. Attribution is a function of the provenance chain, not a post-hoc claim that must be reconstructed from mutable logs.

Content-addressed identity. Each receipt's identifier is derived from its content hash — not from a sequential record number or a timestamp. This means two receipts with identical content produce identical identifiers, and any modification to a receipt's contents changes its identifier. Tampering is structurally detectable, because tampering changes the hash, which changes the identity, which invalidates any subsequent receipt that references the altered receipt as a parent.

> **Provenance Gap**
>
> A model that cannot produce a provenance receipt for its output is making an unverifiable claim about its own behavior. Unverifiable claims are not governance inputs — they are noise.

The community debate around "AI transparency" is, at its structural core, a provenance problem. The demand for transparency is a demand for verifiable chain-of-custody information: who produced this output, under what conditions, using what inputs, through what process, and with what tool calls intervening. Phrasing this as a transparency problem leads to policy responses — disclosure requirements, model cards, system cards, audit reports — that address the demand without satisfying the technical requirement. A model card describes a model's properties at training time. It says nothing about the specific session, context, and input conditions that produced a specific output. Solving the provenance problem requires infrastructure — a receipt ledger, a content-addressed chain of custody — not guidelines. Guidelines describe what should happen. Infrastructure enforces what does happen.

---

5. Prediction Error as Data: Drift Is Not Noise

The field's dominant treatment of prediction error and behavioral drift is as a problem to be minimized — a signal-to-noise ratio to be improved through better prompting, larger models, or more alignment tuning. This framing is not merely incomplete. It is an active waste of some of the most information-dense data available in the agentic AI ecosystem. The Execution Truth framework inverts this treatment: drift is a first-class data type, and an agentic system that discards its drift observations is discarding its behavioral ground truth.

The analogy to regression analysis is precise. In a linear regression, the residuals — the differences between predicted and observed values — are not noise to be discarded after model fitting. They are the diagnostic surface of the model. They reveal whether the model's assumptions are satisfied, where it is systematically biased, and what signal remains in the data that the model has not captured. Discarding residuals after fitting is not neutral — it destroys information. An AI system that discards its prediction errors after generation is making the same mistake at larger scale and with more consequential downstream effects.

5.1 Drift as Training-Grade Signal

When a model's output for a preregistered prompt diverges from a prior execution receipt, that delta is a labeled, attributed, timestamped behavioral observation. It has properties that make it particularly valuable as training data: it is naturally contrastive (two outputs for the same input, produced at different times under potentially different model states), it is attributed (the vendor, model version tag, and session context are encoded in the receipt), it is timestamped with causal ordering (the earlier receipt is the baseline, the later receipt is the observation), and it is structured (the delta can be quantified using semantic distance metrics rather than requiring human annotation).

Drift signals encode information that is not available from any other source. They reveal: how a model's behavior changes between undeclared updates, which prompt structures are fragile versus robust across model versions, where capability gains in one domain co-occur with regressions in another, and how multi-step reasoning chains amplify or attenuate distributional shifts at the step level. This information is not available from static benchmarks, which measure a model at a fixed point in time without tracking behavioral evolution. It is not available from vendor release notes, which describe intended changes without characterizing actual behavioral deltas. It is only available from a structured, longitudinal drift corpus built through the preregistration → execution → checksum → rerun lifecycle.

5.2 The Drift Corpus Architecture

The drift corpus is the artifact produced by operating the Execution Truth lifecycle over time. Each delta — a preregistered expected output class compared against an actual checksummed output — is stored as a structured record with the following fields: drift magnitude (quantified as semantic distance between expected and actual output embeddings), drift direction (capability gain, regression, or lateral shift — a change in capability domain without clear gain or loss), session metadata (session ID, timestamp, vendor, model version tag), and prompt family identifier (enabling cross-session aggregation across prompts of the same structural type).

Over the full 60-night battery, this corpus is intended to become a behavioral time series — a drift timeline — that characterizes a vendor's model evolution with empirical precision that no vendor-provided documentation can match. The vendor can describe what they intended to change. The drift timeline describes what actually changed, in what direction, with what magnitude, and in what prompt domains. As of this writing the corpus reflects the five completed nights; the full timeline accrues through early December 2026 and will be published with its receipts when the battery closes.

Drift Type	Detection Method	Governance Action	Training Data Value
Capability Gain	Output exceeds expected class in evaluated capability dimension; semantic distance positive in gain direction.	Update preregistration baseline for affected prompt family. Log as model evolution event.	High. Positive contrastive pair for capability tracking.

Drift Type	Detection Method	Governance Action	Training Data Value
Capability Regression	Output falls below expected class threshold; prior receipt for same prompt family outperforms current execution.	Quarantine affected prompt family. File drift event with baseline reference. Highest remediation priority.	High. Critical negative pair for regression hardening.
Infrastructure Artifact	Drift pattern is session-correlated with non-prompt factors: time of day, API latency, rate-limiting state.	Separate from model-level drift. Log as infrastructure noise. Do not include in model behavioral corpus.	Low for model training. High for infrastructure diagnosis.
Context Bleed	Output reflects context from a prior session not declared in current preregistration.	Immediate session isolation audit. Context bleed is a control plane failure, not a drift event.	High. Context bleed examples are critical negative training cases for session isolation enforcement.

The governance implication is direct: a system that logs drift is a system that can prove behavioral change. A system that does not log drift cannot distinguish a model update from a prompt anomaly from an infrastructure artifact. These are not equivalent uncertainty classes — they require different remediation, different attribution, and different governance responses. Collapsing them into undifferentiated "model behavior" is a governance failure that the drift corpus architecture is specifically designed to prevent.

---

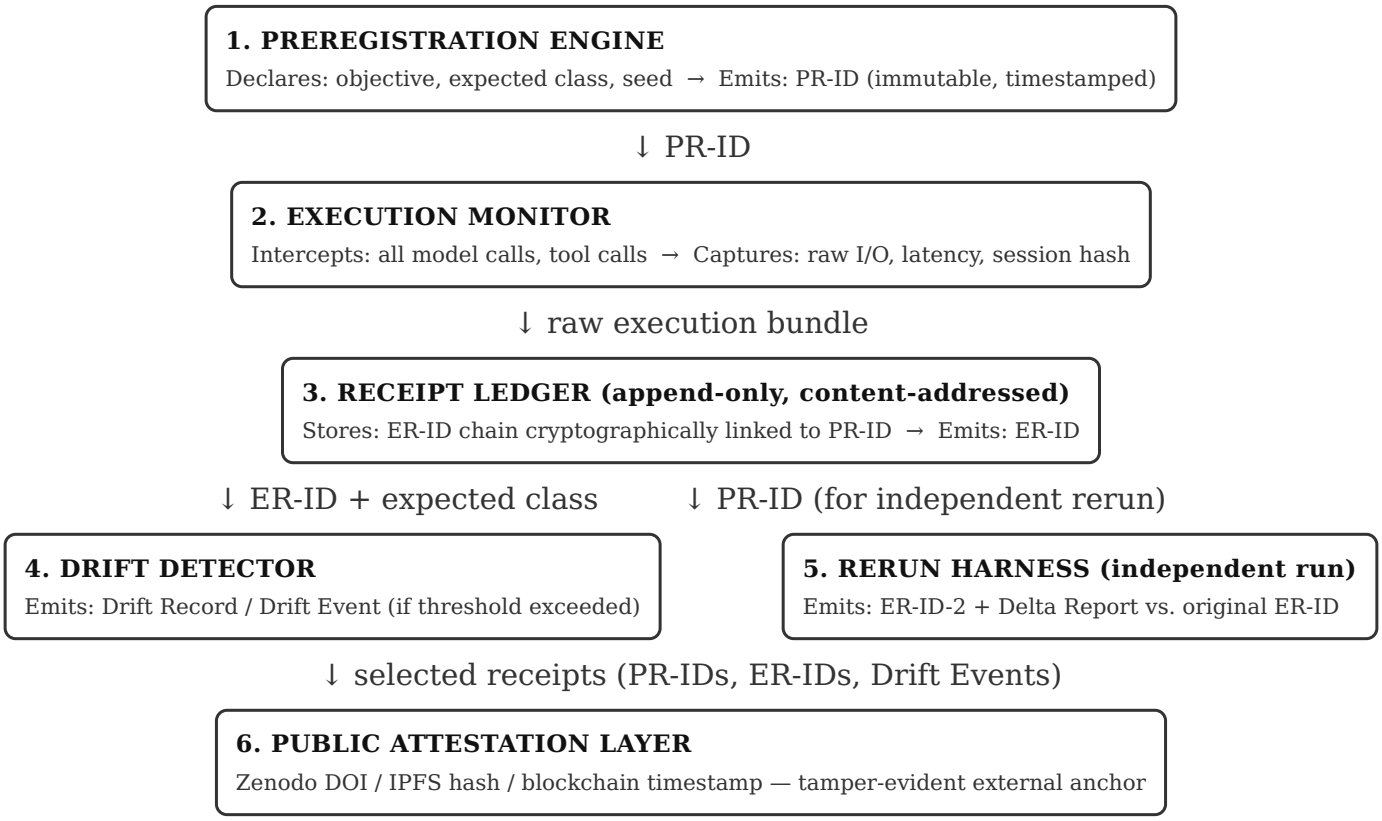
6. The Deterministic Cage: Architecture of the Control Plane

> **Implementation status.** This section is a design specification, partially implemented. The tested implementation to date is a deterministic containment gate operating in a local mock-execution integration, receipted across four LLM engines with zero side effects on every rejected action (CCR-1, DOI 10.5281/zenodo.23288189; CCR-2, DOI 10.5281/zenodo.23288526; containment program closeout, public ledger entries 0045–0049). The deterministic resolution layer is measured in the Dual Mesh Benchmark (DOI 10.5281/zenodo.23303081). The full six-component runtime described below is the target architecture. Statements in this section describe the design's enforcement properties as specified, not as production-proven claims.

The deterministic cage is the runtime architecture that enforces Execution Truth. It is not a monitoring wrapper applied around existing agentic systems. It is a control plane layer that operates between the orchestration layer and the model API surface, enforcing the simulation/execution boundary at every interaction point. The cage earns its name: it constrains the behavior space of the agentic session to the parameters declared at preregistration, and it produces a verifiable record of every deviation from those parameters.

The architecture comprises six components. Each has a defined interface, a defined input/output contract, and a defined receipt type. Together they implement the preregistration → execution → checksum → public rerun lifecycle as a runtime-enforced system rather than a methodological aspiration.

FIGURE 2



Component 1: Preregistration Engine. The Preregistration Engine accepts a declared test objective, an expected output class (defined as a structured schema against which outputs can be evaluated), and a checksum seed (a deterministic value used to initialize the session hash chain). It validates that all required fields are present and well-formed before any model interaction is permitted. On validation, it generates a Preregistration Receipt (PR-ID): a content-addressed artifact containing all declared parameters plus a UTC timestamp. The PR-ID is immutable from the moment of generation. No modification to test parameters is permitted after the PR-ID is issued — modifications require a new preregistration with a reference to the prior PR-ID as a superseded parent. The Preregistration Engine enforces the fundamental control plane contract: parameters are declared before execution begins, not reconstructed after execution completes.

Component 2: Execution Monitor. The Execution Monitor intercepts all model invocations and tool calls within a preregistered session. It captures raw I/O at every step: the full input context hash, the full model output, any tool call identifiers with input parameters and returned states, and latency metadata at each invocation boundary. The Execution Monitor enforces session isolation — it detects and flags any context that was not declared in the originating PR-ID, preventing context bleed between preregistered runs. Context bleed is not treated as a drift event; it is treated as a control plane failure and triggers an immediate isolation audit separate from the normal receipt flow.

Component 3: Receipt Ledger. The Receipt Ledger stores all execution receipts in an append-only log. The append-only constraint is architectural, not policy: the ledger's data structure does not support in-place modification. Every receipt is content-addressed — its identifier is the cryptographic hash of its content, making modification structurally detectable. Receipt chains are cryptographically linked through parent references: each receipt includes the identifier of the prior receipt in the same session chain, creating a tamper-evident sequence in which altering any receipt invalidates all subsequent receipts that reference it. Within the cage, the Receipt Ledger is defined as the authoritative source of execution truth — no action claim is valid unless a corresponding receipt exists in the ledger.

Component 4: Drift Detector. The Drift Detector compares each new Execution Receipt (ER-ID) against the declared expected output class in the originating PR-ID and against any prior ER-IDs in the same prompt family. It computes a structured drift record for every session: drift magnitude as a semantic distance metric between expected and actual output representations, drift direction classification against the taxonomy defined in Section 5, and a confidence score based on prompt family sample size. Drift records below a configurable significance threshold are archived but not escalated. Drift records at or above threshold are escalated to the receipt ledger as Drift Events — first-class records that trigger downstream governance actions.

Component 5: Rerun Harness. The Rerun Harness accepts a prior PR-ID and reruns the identical preregistered test parameters against the current model state. It is designed to be operated by an independent party — someone with access to the PR-ID but without access to the original session context, session logs, or prior execution receipts. The Rerun Harness produces a new ER-ID from the independent execution and a comparative delta report — a structured comparison of the original ER-ID and the new ER-ID that characterizes behavioral change between the two execution points. The delta report is the audit artifact: it is the evidence that an independent party can produce by running the preregistered test, and it is the mechanism by which the governance cycle is externally validated.

Component 6: Public Attestation Layer. The Public Attestation Layer exposes a defined subset of receipts — PR-IDs, ER-IDs for designated public tests, and Drift Event records — to an external audit surface. The external anchor may be a Zenodo DOI issued against a receipt bundle, an IPFS content address published to a public gateway, or a blockchain timestamp (a hash commitment to a public ledger that does not require publishing receipt content — only its hash). The Public Attestation Layer creates a tamper-evident external anchor for the receipt chain: even if the internal receipt ledger were compromised, the external anchor proves that a specific receipt existed at a specific time and had specific content.

The design principles borrowed from blockchain execution environments are explicit throughout this architecture. **Append-only:** receipts are never overwritten, only superseded by new receipts with parent references. **Content-addressed:** identity is derived from content, not from location or mutable label. **Consensus-required:** within the cage, an action is not "executed" until a receipt exists in the ledger — no receipt

means no execution, regardless of what the model asserts. **Deterministic replay:** any execution should be replayable from its PR-ID — the preregistration defines the inputs completely enough that an independent party can reproduce the test and compare receipts.

The contrast with current AI infrastructure is precise and consequential. Today's AI platforms offer logging. Logging is human-readable observation of activity: it records what appeared to happen, in formats designed for human review, without cryptographic commitment to content, without content-addressing, and without the structural properties that make independent verification possible. Receipts are machine-verifiable commitments: they prove that a specific event occurred, under specific conditions, at a specific time, and can be independently verified by any party with access to the PR-ID and a compatible Rerun Harness. The difference between logging and receipts is not a matter of log verbosity or retention policy. It is a categorical architectural distinction.

---

7. Implications for the Field: What Changes When You Add a Control Plane

The Execution Truth framework is not a marginal improvement to existing AI governance practice. Adding a control plane to agentic AI changes the nature of governance, the tractability of liability, the economics of auditing, and — plausibly — the competitive structure of the industry itself.

Governance becomes runtime enforcement. Current AI governance operates on assertions: model cards, policy documents, evaluation reports, red-team summaries. These are all statements about behavior, produced after the fact, under conditions the governor did not control. A control plane converts governance into a runtime property: the system either permitted an action against a validated preregistration, or it did not. Governance questions stop being interpretive and become queryable.

Liability becomes tractable. When an agentic system takes an action with real-world consequences — sends a wire, files a document, modifies a production system — the question of attribution is currently unanswerable in a rigorous sense. Logs can be reconstructed; session context cannot. With a receipt chain, attribution is not a reconstruction — it is a query. The receipt for the damaging action identifies the model version tag, the session ID, the input hash, the tool calls with their sub-receipts, and the timestamp. Attribution is structural, not inferential.

Auditing becomes deterministic. Third-party AI audits currently depend on access to model internals, vendor-provided documentation, and samples of model behavior collected under conditions that the auditor did not fully control. Findings from such audits are interpretive: they characterize behavior under observed conditions without being able to guarantee that the conditions are representative or that the observed behavior would recur. With the Rerun Harness, auditing is deterministic: the auditor receives the PR-ID for a designated set of preregistered tests, reruns them independently,

and compares the resulting ER-IDs against the baseline ER-IDs in the public attestation layer. Audit findings are not interpretive — they are delta reports. The audit either finds that behavior is consistent with the baseline within declared thresholds, or it documents the specific receipts where it is not.

Competition on execution quality. Enterprise AI procurement is currently evaluated primarily on capability benchmarks and vendor assurances. As agentic AI penetrates higher-stakes operational contexts — legal, financial, medical, infrastructure — we expect the procurement criteria to shift. An enterprise operating in a regulated environment cannot indefinitely accept an AI system that cannot produce execution receipts for its actions. We expect the Execution Truth framework, or something structurally equivalent to it, to become a procurement requirement in those contexts. Vendors who build receipt infrastructure into their platforms will be differentiated. We expect vendors who cannot produce execution receipts to be disqualified in those contexts — not because of any regulatory mandate, but because the enterprise risk calculus of operating a critical workflow on an unreceipted system is no longer acceptable once the alternative exists.

The research ecosystem benefits. Cross-vendor behavioral datasets built from structured drift corpora would enable a class of meta-research on model evolution that is currently impossible. Researchers cannot currently study how a model's behavior changes across updates, how capability gains and regressions co-occur across domains, or how behavioral drift patterns differ across vendors, because there is no shared infrastructure for capturing this data systematically. The drift corpus architecture creates that infrastructure as a natural byproduct of the governance lifecycle.

The independent researcher advantage. Perhaps the most significant implication is the democratization of empirical AI research. An independent researcher without access to model internals, without a research agreement with a vendor, and without institutional affiliation can still build an authoritative behavioral record of agentic AI systems by operating the preregistration → execution → checksum → public rerun lifecycle. The battery described in this paper — five nights complete, fifty-five in flight — is an existence proof in progress. The control plane does not require model access. It requires methodological discipline and the infrastructure to enforce it. This is a domain in which independent researchers have a structural advantage over large institutions: the methodology is the asset, and the methodology is not gatekept.

---

8. Positioning and Prior Art

The Execution Truth framework builds on a set of well-established prior contributions across computer science, distributed systems, and research methodology. Acknowledging this prior art precisely — without overstating convergence — is important for situating the framework's contribution accurately.

Blockchain execution determinism. The Ethereum Virtual Machine and Solana's runtime established the receipts-as-truth primitive at production scale. In both systems, a

transaction receipt is not an optional audit artifact — it is the proof of execution. State transitions that lack a confirmed receipt did not happen, regardless of what any participant asserts. The EVM's deterministic execution model, in which identical inputs produce identical state transitions across all nodes, is the direct ancestor of the deterministic replay principle in the Execution Truth framework. The contribution here is the application of this primitive to the agentic AI task layer — a layer that did not exist when these systems were designed and that has different execution semantics (probabilistic model outputs vs. deterministic bytecode) requiring adapted mechanisms.

Reproducible research and preregistration. The Open Science Foundation's preregistration infrastructure and the Registered Reports publication format established preregistration as a scientific norm in empirical research. The core insight — that declaring hypotheses and methods before collecting data is a structural safeguard against post-hoc rationalization and selective reporting — translates directly to the agentic AI context. The Execution Truth framework operationalizes preregistration at the session level: every preregistered session is the AI equivalent of a Registered Report, with the PR-ID serving as the pre-data-collection declaration and the ER-ID serving as the registered outcome. The contribution is the extension of this norm from hypothesis-level preregistration to per-session, per-interaction preregistration enforced at runtime.

Formal verification in distributed systems. The formal methods community — TLA+, Coq, Isabelle, and their ecosystem — established that correctness proofs for distributed systems require execution traces, not just logical models. A system can be proven correct under its formal specification and still fail in deployment because the specification does not capture all relevant environmental assumptions. Execution traces are the empirical complement to formal proofs: they record what actually happened in a specific execution environment, enabling comparison between the formal model and operational reality. The Execution Truth receipt chain is an execution trace architecture applied to the agentic AI layer.

AI safety and mechanistic interpretability. The mechanistic interpretability research program — activation patching, circuit analysis, representation engineering — focuses on understanding model behavior at the weight and activation level. This is a complementary concern, not a competing one. Mechanistic interpretability asks: what is happening inside the model that produces this behavior? Execution Truth asks: what did the model produce, under what conditions, and can we verify it independently? The two questions operate at different layers of the stack. Mechanistic interpretability operates at the weight level, requiring access to model internals. Execution Truth operates at the API surface, requiring only access to model outputs and the infrastructure to receipt them. Organizations without model access — which is the majority of the field — can implement Execution Truth without any access to model internals. The two approaches are additive.

What Execution Truth adds, and what none of the above prior art addresses directly, is a receipts-based, externally verifiable control plane that operates at the agentic task layer — the layer at which an LLM is orchestrated to take multi-step actions in the world on behalf of a user or system. This layer did not exist as a deployment target when the prior art was developed. The prior art establishes the constituent principles. The Execution

Truth framework assembles those principles into an architecture for a new problem domain.

8.5 Receipted Evidence Base

Every empirical claim in this paper is anchored to a public, DOI-stamped record. The evidence base:

Artifact	DOI / Location
Master thesis — Deterministic Containment of Probabilistic Intelligence: The Squirrel OS Journey	10.5281/zenodo.23303240
Dual Mesh Benchmark: Preregistered Latency and Integrity Measurements for Deterministic Anomaly-to-Playbook Resolution	10.5281/zenodo.23303081
Cage Containment Receipt 1 (CCR-1): Deterministic Gate Blocks Invalid LLM Proposals Before Execution Across Four Engines	10.5281/zenodo.23288189
Cage Containment Receipt 2 (CCR-2): A Live LLM Proposes an Invalid Action Under Explicit Instruction; the Deterministic Gate Blocks It Before Execution	10.5281/zenodo.23288526
Deterministic Caging of Probabilistic Large Language Models: Architecture, Field Receipt, and a Drift-Audit Case Study	10.5281/zenodo.23284729
Receipts Over Genius: An Audited Account of an 18-Month, AI-Assisted, Solo Invention Cycle (drift specimen catalog)	10.5281/zenodo.23267773
SDR-CL1: First Model-Attribution Replay of a Registered Substrate Determinism Gauntlet Across LLMs	10.5281/zenodo.23274887
SDR-CL2: Third-Vendor Model-Attribution Replay (Microsoft 365 Copilot)	10.5281/zenodo.23284295
Deterministic Runtime Architecture (Public Edition v3.1) — enterprise architecture deck	10.5281/zenodo.23292636
Public receipts ledger (nightly battery entries; containment program closeout 0045–0049; acknowledgments 0050–0051)	github.com/LLong2026/four-horsemen
Research portal	dsos-science-scope.base44.app
Research community (DCAI)	zenodo.org/communities/dcai

---

9. Conclusion: The Receipt Is the Contract

The central claim of this paper is simple and deliberately unhedged: an AI system that cannot produce a verifiable execution receipt for a claimed action has not acted. It has simulated acting. The distinction matters because the field has reached the point where agentic AI systems are being deployed in contexts — financial operations, legal workflows, healthcare coordination, infrastructure management — where the difference between acting and simulating acting has real consequences. Systems that cannot distinguish the two are not ready for those contexts, regardless of their benchmark performance on capability evaluations.

The ongoing 60-night cross-vendor battery provides the empirical grounding for this claim to date. In the configurations tested, no native receipt infrastructure existed. No configuration tested provided a mechanism by which an external party could independently verify that a specific output was produced under specific conditions and rerun the identical test to compare results. The governance gap documented in this paper is not hypothetical. It is operational, it is cross-vendor within the tested set, and it is observable by any practitioner who applies the preregistration methodology to their own agentic AI workflows.

The path forward is clear. Standardize the execution receipt format — define the minimum required fields, the content-addressing scheme, and the chain-linking mechanism as an open specification that any vendor can implement. Require preregistration for public benchmarks — a benchmark result without a PR-ID is an assertion, not a finding. Treat drift as data — build the infrastructure to capture, structure, and attribute behavioral deltas rather than discarding them. Build the control plane — implement the six-component deterministic cage architecture as runtime infrastructure, not as a monitoring afterthought.

None of these steps require solving the unsolved problems of AI alignment or mechanistic interpretability. They require applying engineering disciplines that the broader computer science community has mastered to a domain that has resisted applying them. The resistance is not technical — it is a matter of architectural priority. Execution Truth is an argument that the priority must change.

> **Core Doctrine**
>
> The receipt is the contract. The checksum is the signature. The rerun is the proof.

APPENDIX A — Lifecycle Notation Reference

The following notation defines the formal structure of the Execution Truth lifecycle. Each stage is specified by its required inputs, produced outputs, and the receipt type generated. This notation is intended as a reference for implementers building compatible control plane infrastructure.

Stage 1: PREREGISTER

Field	Type	Description	Required
test_objective	String	Declarative statement of what the session is testing. Must be specific enough to evaluate against a defined output class.	Required
expected_output_class	Schema	Structured definition of the expected output — type, format, capability domain, and evaluation criteria.	Required
checksum_seed	Bytes		Required

Field	Type	Description	Required
		Deterministic value used to initialize the session hash chain. Must be unique per preregistration.	
parent_pr_id	Hash	Reference to a prior PR-ID if this preregistration supersedes or extends a prior one.	Optional
Output: PR-ID	Receipt	Content-addressed hash of all input fields plus UTC timestamp. Immutable from generation. Locked before any model interaction.	—

Stage 2: EXECUTE

Field	Type	Description	Required
pr_id	Hash	The PR-ID locking the session parameters. Execution Monitor validates PR-ID before permitting any model invocation.	Required
model_invocation	API Call	The live model API call(s) within the session scope. All calls are intercepted and logged by the Execution Monitor.	Required
input_context_hash	Hash	Cryptographic hash of the full input context at the moment of each model invocation.	Auto (captured by Monitor)
raw_output	String	Full model output, untruncated, at each invocation step.	Auto
tool_call_log	List[SubReceipt]	Ordered list of tool calls with identifier, input parameters, returned state, latency.	Auto
latency_metadata	Struct	Latency at each invocation boundary, in milliseconds.	Auto

Stage 3: CHECKSUM

Field	Type	Description	Required
execution_bundle	Struct	All fields captured by the Execution Monitor, plus PR-ID and model version tag as declared by vendor API.	Required
Output: ER-ID	Receipt	Content-addressed hash of the complete execution bundle. Stored in Receipt Ledger. Identity derived from content — any modification changes the hash.	—

Stage 4: PUBLIC RERUN

Field	Type	Description	Required
pr_id	Hash	The original PR-ID provided to the independent party. No access to original session context or prior ER-ID is permitted.	Required
independent_execution	API Call	Independent party reruns the identical preregistered test parameters through their own Rerun Harness instance.	Required

Field	Type	Description	Required
er_id_2	Receipt	New execution receipt from the independent run. Compared against the original ER-ID.	Auto
delta_report	Struct	Structured comparison of ER-ID vs. ER-ID-2: drift magnitude, drift direction, semantic distance, and classification per taxonomy.	Auto
attestation_anchor	External Hash	Zenodo DOI, IPFS hash, or blockchain timestamp anchoring the delta report to a tamper-evident external record.	Recommended

APPENDIX B — Drift Classification Reference Table

The following table provides the full taxonomy of drift types recognized by the Execution Truth framework's Drift Detector component. Detection criteria, required governance responses, training data disposition, and escalation thresholds are specified for each type. Implementers should treat this taxonomy as a minimum viable classification schema; domain-specific extensions are encouraged provided they maintain the structured record format required for drift corpus aggregation.

Drift Type	Detection Criteria	Escalation Threshold	Governance Response	Training Data Disposition
Capability Gain	Actual output exceeds expected class in declared capability dimension. Semantic distance is positive in the gain direction relative to baseline ER-ID.	Any gain exceeding one capability tier above declared expected class.	Update PR baseline for affected prompt family. Log as model evolution event with vendor attribution. No remediation required unless gain disrupts downstream dependencies.	Include in corpus as positive contrastive pair. Label: [baseline_er_id, gain_er_id, capability_dimension, gain_magnitude]. High training value for capability tracking.
Capability Regression	Actual output falls below expected class. Prior ER-ID for same prompt family outperforms current execution on declared evaluation criteria.	Any regression below expected class threshold. Zero tolerance in designated critical prompt families.	Immediate alert. Quarantine affected prompt family. File vendor attribution record with ER-ID evidence. Do not update baseline until regression is explained and resolved.	Include in corpus as critical negative pair. Label: [prior_er_id, regression_er_id, affected_domain, regression_magnitude]. Highest training data priority class.
Lateral Shift	Output changes domain, style, or structural format without	Lateral shift exceeding 15% semantic distance from	Document as behavioral evolution. Monitor for prompt	Include in corpus with direction label: [lateral_shift]. Medium training value. Useful for

Drift Type	Detection Criteria	Escalation Threshold	Governance Response	Training Data Disposition
	measurable gain or loss in declared capability dimension. Semantic distance is non-zero but undirected.	baseline representation.	fragility effects in downstream sessions. No immediate remediation unless shift affects declared output class schema compliance.	cross-domain sensitivity analysis and prompt hardening benchmarks.
Prompt Fragility	High variance across multiple reruns of identical preregistered prompt without any declared model update between sessions. Variance exceeds intra-session sampling noise baseline.	Coefficient of variation exceeding 20% across three or more independent reruns within a 30-session window.	Flag prompt family for structural redesign. Document as governance liability. Fragile prompts must not be used in production agentic workflows until hardened and re-validated against a new PR-ID.	Include full rerun set as fragility corpus. Label: [high_variance, prompt_family_id, variance_metric]. High value for prompt hardening research and adversarial robustness evaluation.
Infrastructure Artifact	Drift pattern correlates with non-prompt factors: time of day, API latency percentile, concurrent request volume, rate-limiting state. Pattern is not reproducible across independent execution environments.	Any drift event where infrastructure correlation coefficient exceeds model-level attribution confidence.	Separate from model-level drift corpus. Log in infrastructure artifact register. Notify platform operations if pattern indicates systematic degradation. Do not file vendor model attribution record.	Exclude from model behavioral corpus. Include in infrastructure diagnosis dataset. Label: [infrastructure_artifact, correlation_factor, session_metadata]. Low model training value; high ops value.
Context Bleed	Output reflects context, facts, preferences, or session state from a prior session not declared in the current preregistration. Execution Monitor flags undeclared context at capture.	Any confirmed context bleed instance, regardless of magnitude. Zero-tolerance event class.	Immediate session isolation audit. Do not issue standard ER-ID for the contaminated session. Issue a Contamination Record (CR-ID) referencing the detected bleed source. Escalate to platform	Include contamination examples in critical negative corpus. Label: [context_bleed, source_session_id (if identifiable), bleed_content_hash]. Critical training data for session isolation enforcement and privacy boundary validation.

Drift Type	Detection Criteria	Escalation Threshold	Governance Response	Training Data Disposition
			security if source cannot be identified.	
Undeclared Model Update	Systematic drift across multiple prompt families within a single session window, without any vendor-declared model update. Pattern is cross-prompt, not prompt-specific, and persists across independent reruns.	Systematic drift affecting three or more distinct prompt families within a 72-hour session window.	Issue cross-family Drift Alert. Request vendor confirmation of model state. If vendor confirms no update, log as unexplained systematic drift and escalate to independent attestation layer for external anchoring.	Include in vendor behavioral evolution corpus with explicit undeclared_update flag. Label: [systematic_drift, affected_families, session_window, vendor_attribution]. High value for vendor accountability research and cross-vendor comparative analysis.

Leon Calvin Long II | SquirlOS Technologies, LLC | Execution Truth Doctrine Series |
October 2026 Independent Research — Preprint. Not Peer Reviewed. Licensed CC BY 4.0 — Attribution Required. Correspondence: SquirlOS Technologies, LLC | Spicewood, TX, United States

> This software is a prototype and is provided for educational and research purposes only. It is not intended for production use, commercial deployment, or safety-critical environments. All systems are experimental and may contain defects on them.